

Code refactoring

In computer programming and software design, **code refactoring** is the process of restructuring existing source code—changing the *factoring*—without changing its external behavior. Refactoring is intended to improve the design, structure, or implementation of the software (its *non-functional* attributes), while preserving its functionality. Potential advantages of refactoring may include improved code readability and reduced complexity; these can improve the source code's maintainability and create a simpler, cleaner, or more expressive internal architecture or object model to improve extensibility. Another potential goal for refactoring is improved performance; software engineers face an ongoing challenge to write programs that perform faster or use less memory. Despite the availability of metrics intended to assess such attributes, developers often do not rely on them when making refactoring decisions due to concerns about its practical usefulness.^[1]

Typically, refactoring applies a series of standardized basic *micro-refactorings*, each of which is (usually) a tiny change in a computer program's source code that either preserves the behavior of the software, or at least does not modify its conformance to functional requirements. Many development environments provide automated support for performing the mechanical aspects of these basic refactorings. If done well, code refactoring may help software developers discover and fix hidden or dormant bugs or vulnerabilities in the system by simplifying the underlying logic and eliminating unnecessary levels of complexity. If done poorly, it may fail the requirement that external functionality not be changed, and may thus introduce new bugs.

By continuously improving the design of code, we make it easier and easier to work with. This is in sharp contrast to what typically happens: little refactoring and a great deal of attention paid to expediently add new features. If you get into the hygienic habit of refactoring continuously, you'll find that it is easier to extend and maintain code.

—Joshua Kerievsky, *Refactoring to Patterns*^[2]

Motivation

Refactoring is usually motivated by noticing a code smell.^[3] For example, the method at hand may be very long, or it may be a near duplicate of another nearby method. Once recognized, such problems can be addressed by *refactoring* the source code, or transforming it into a new form that behaves the same as before but that no longer "smells".

For a long routine, one or more smaller subroutines can be extracted. For duplicate routines, the duplication can be removed and replaced with one shared function. Failure to perform refactoring can result in accumulating technical debt; on the other hand, refactoring is one of the primary means of repaying technical debt.^[4]

Benefits

There are two general categories of benefits to the activity of refactoring.

1. Maintainability. It is easier to fix bugs because the source code is easy to read and the intent of its author is easy to grasp.^[5] This might be achieved by reducing large monolithic routines into a set of individually concise, well-named, single-purpose methods. It might be achieved by moving a method to a more appropriate class or by removing misleading comments.
2. Extensibility. It is easier to extend the capabilities of the application if it uses recognizable design patterns, and it provides some flexibility where none before may have existed.^[2]

Performance engineering can remove inefficiencies in programs, known as software bloat, arising from traditional software-development strategies that aim to minimize an application's development time rather than the time it takes to run. Performance engineering can also tailor software to the hardware on which it runs, for example, to take advantage of parallel processors and vector units.^[6]

Timing and responsibility

There are two possible times for refactoring.

1. Preventive refactoring – the original developer of the code makes the code more robust when it is still free of smells to prevent the formation of smells in the future.^[7]
2. Corrective refactoring – a subsequent developer performs refactoring to correct code smells as they occur.^[7]

A method that balances preventive and corrective refactoring is "shared responsibility for refactoring". This approach splits the refactoring action into two stages and two roles. The original developer of the code just prepares the code for refactoring, and when the code smells form, a subsequent developer carries out the actual refactoring action.^[7]

Challenges

Refactoring requires extracting the software system structure, data models, and intra-application dependencies to get back knowledge of an existing software system.^[8] The turnover of teams implies missing or inaccurate knowledge of the current state of a system and about design decisions made by departing developers. Further code refactoring activities may require additional effort to regain this knowledge.^[9] Refactoring activities generate architectural modifications that deteriorate the structural architecture of a software system. Such deterioration affects architectural properties such as maintainability and comprehensibility, which can lead to a complete re-development of software systems.^[10]

Testing

Automatic unit tests should be set up before refactoring to ensure routines still behave as expected.^[11] Unit tests can bring stability to even large refactors when performed with a single atomic commit. A common strategy to allow safe and atomic refactors spanning multiple projects is to store all projects in a single repository, known as monorepo.^[12]

With unit testing in place, refactoring is then an iterative cycle of making a small program transformation, testing it to ensure correctness, and making another small transformation. If at any point a test fails, the last small change is undone and repeated in a different way. Through many small steps the program moves from where it was to where you want it to be. For this very iterative process to be practical, the tests must run very quickly, or the programmer would have to spend a large fraction of their time waiting for the tests to finish. Proponents of extreme programming and other agile software development describe this activity as an integral part of the software development cycle. Refactoring applies to test code as well as production code; empirical studies show that test refactoring primarily targets low-quality test code exhibiting test smells, and improves test code coupling, cohesion, and size, though it does not necessarily improve code coverage.^[13]

Techniques

Here are some examples of micro-refactorings; some of these may only apply to certain languages or language types. A longer list can be found in Martin Fowler's refactoring book^[3] and website.^[14] Many development environments provide automated support for these micro-refactorings. For instance, a programmer could click on the name of a variable and then select the "Encapsulate field" refactoring from a context menu. The IDE would then prompt for additional details, typically with sensible defaults and a preview of the code changes. After confirmation by the programmer, it would carry out the required changes throughout the code.

Static analysis

Static program analysis (called "linting" when performed on less strict interpreted languages) detects problems in a valid but substandard program.

- Program dependence graph - explicit representation of data and control dependencies ^[15]
- System dependence graph - representation of procedure calls between PDG ^[16]
- Cyclometric complexity analysis.
- Software intelligence - reverse engineers the initial state to understand existing intra-application dependencies

Transformations

Transformations modify the syntactic representation of a program. Some modifications alter the semantics or structure of the program in a way that improves its flexibility or robustness. Such modifications require knowledge of the problem domain and intended logic, and thus are infeasible to

automate. Modifications exist that make the program easier to read and modify, but which do not alter the underlying logic of the program; these transformations can be automated.

- Techniques that allow for more abstraction
 - Encapsulate field – force code to access the field with getter and setter methods
 - Generalize type – create more general types to allow for more code sharing
 - Replace type-checking code with state/strategy^[17]
 - Replace conditional with polymorphism^[18]
- Techniques for breaking code apart into more logical pieces
 - Componentization breaks code down into reusable semantic units that present clear, well-defined, simple-to-use interfaces.
 - Extract class moves part of the code from an existing class into a new class.
 - Extract method, to turn part of a larger method into a new method. By breaking down code in smaller pieces, it is more easily understandable. This is also applicable to functions.
- Techniques for improving names and location of code
 - Move method or move field – move to a more appropriate class or source file
 - Rename method or rename field – changing the name into a new one that better reveals its purpose
 - Pull up – in object-oriented programming (OOP), move to a superclass
 - Push down – in OOP, move to a subclass^[14]
- Automatic clone detection^[19]

Hardware refactoring

While the term *refactoring* originally referred exclusively to refactoring of software code, in recent years code written in hardware description languages has also been refactored. The term *hardware refactoring* is used as a shorthand term for refactoring of code in hardware description languages. Since hardware description languages are not considered to be programming languages by most hardware engineers,^[20] hardware refactoring is to be considered a separate field from traditional code refactoring.

Automated refactoring of analog hardware descriptions (in VHDL-AMS) has been proposed by Zeng and Huss.^[21] In their approach, refactoring preserves the simulated behavior of a hardware design. The non-functional measurement that improves is that refactored code can be processed by standard synthesis tools, while the original code cannot. Refactoring of digital hardware description languages, albeit manual refactoring, has also been investigated by Synopsys fellow Mike Keating.^{[22][23]} His target is to make complex systems easier to understand, which increases the designers' productivity.

History

The first known use of the term "refactoring" in the published literature was in a September 1990 article by William Opdyke and Ralph Johnson.^[24] Although refactoring code has been done informally for decades, William Griswold's 1991 Ph.D. dissertation^[25] is one of the first major academic works on refactoring functional and procedural programs, followed by William Opdyke's 1992 dissertation^[26] on

the refactoring of object-oriented programs,^[27] although all the theory and machinery have long been available as program transformation systems. All of these resources provide a catalog of common methods for refactoring; a refactoring method has a description of how to apply the method and indicators for when you should (or should not) apply the method.

Martin Fowler's book *Refactoring: Improving the Design of Existing Code* is the canonical reference.

The terms "factoring" and "factoring out" have been used in this way in the Forth community since at least the early 1980s. Chapter Six of Leo Brodie's book *Thinking Forth* (1984)^[28] is dedicated to the subject.

In extreme programming, the Extract Method refactoring technique has essentially the same meaning as factoring in Forth: to break down a "word" (or function) into smaller, more easily maintained functions.

Refactorings can also be reconstructed^[29] posthoc to produce concise descriptions of complex software changes recorded in software repositories like Git.

Tools

Many software development tools have features for refactoring. Examples include:

Android Studio

Supports refactoring Java and C++.

AppCode

Supports refactoring Objective-C, C and C++.

Delphi

Supports refactoring Object Pascal.

DMS Software Reengineering Toolkit

Supports large-scale refactoring for C, C++, C#, COBOL, Java, PHP and other languages.

Eclipse

Via plugins, supports refactoring Java and to a lesser extent C++, PHP, Ruby and JavaScript

IntelliJ IDEA

Supports refactoring Java.

JDeveloper

Supports refactoring Java.

NetBeans

Supports refactoring Java.

PhpStorm

Supports refactoring PHP.

PyCharm

Supports refactoring Python.

PyDev

Supports refactoring Python.

Smalltalk

Most dialects include powerful refactoring tools. Many use the original refactoring browser produced in the early '90s by Ralph Johnson.

Visual Assist

Supports refactoring C# and C++.

Visual Studio

Supports refactoring .NET and C++.

Visual Studio Code

Supports refactoring many languages via plugins

WebStorm

Supports refactoring JavaScript.

Wing IDE

Supports refactoring Python.

Xcode

Supports refactoring C, Objective-C, and Swift.^[30]

Qt Creator

Supports refactoring C++, Objective-C and QML.^[31]

See also

- Amelioration pattern
- Code review
- Database refactoring
- Decomposition (computer science)
- Modular programming
- Obfuscated code
- Prefactoring
- Rewrite (programming)
- Separation of concerns
- Software peer review
- Test-driven development

References

1. Tempero, Ewan; Ralph, Paul (2026-03-16). "Making Software Metrics Useful". arXiv:2603.16012 (<https://arxiv.org/abs/2603.16012>) [cs.SE (<https://arxiv.org/archive/cs/SE>)].
2. Kerievsky, Joshua (2004). *Refactoring to Patterns*. Addison Wesley.
3. Fowler, Martin (1999). *Refactoring. Improving the Design of Existing Code* (https://archive.org/details/isbn_9780201485677/page/63). Addison-Wesley. pp. 63ff (https://archive.org/details/isbn_9780201485677/page/63). ISBN 978-0-201-48567-7.
4. Suryanarayana, Girish (November 2014). *Refactoring for Software Design Smells*. Morgan Kaufmann. p. 258. ISBN 978-0-12-801397-7.
5. Martin, Robert (2009). *Clean Code*. Prentice Hall.
6. Leiserson, Charles E.; Thompson, Neil C.; Emer, Joel S.; Kuszmaul, Bradley C.; Lampson, Butler W.; Sanchez, Daniel; Schardl, Tao B. (2020). "There's plenty of room at the Top: What will drive computer performance after Moore's law?" (<https://doi.org/10.1126%2Fscience.aam9744>). *Science*. **368** (6495) eaam9744. Bibcode:2020Sci...368m9744L (<https://ui.adsabs.harvard.edu/abs/2020Sci...368m9744L>). doi:10.1126/science.aam9744 (<https://doi.org/10.1126%2Fscience.aam9744>). PMID 32499413 (<https://pubmed.ncbi.nlm.nih.gov/32499413>).
7. Fraivert, Dov; Lorenz, David H. (2022). "Language Support for Refactorability Decay Prevention". *Proceedings of the 21st ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*. pp. 122–134. doi:10.1145/3564719.3568688 (<https://doi.org/10.1145%2F3564719.3568688>). ISBN 978-1-4503-9920-3.

8. Haendler, Thorsten; Neumann, Gustaf (2019). "A Framework for the Assessment and Training of Software Refactoring Competences". *Proceedings of the 11th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. pp. 307–316. doi:10.5220/0008350803070316 (<https://doi.org/10.5220/0008350803070316>). ISBN 978-989-758-382-7. S2CID 204754665 (<https://api.semanticscholar.org/CorpusID:204754665>).
9. Nassif, Matthieu; Robillard, Martin P. (November 2017). *Revisiting Turnover-Induced Knowledge Loss in Software Projects*. 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). pp. 261–272. doi:10.1109/ICSME.2017.64 (<https://doi.org/10.1109/2FICSME.2017.64>). ISBN 978-1-5386-0992-7. S2CID 13147063 (<https://api.semanticscholar.org/CorpusID:13147063>).
10. van Gorp, Jilles; Bosch, Jan (March 2002). "Design erosion: problems and causes". *Journal of Systems and Software*. **61** (2): 105–119. doi:10.1016/S0164-1212(01)00152-2 (<https://doi.org/10.1016/2FS0164-1212/2801/2900152-2>).
11. Fowler, Martin (1999). *Refactoring: improving the design of existing code* (https://archive.org/details/isbn_9780201485677). Reading, MA: Addison-Wesley. ISBN 978-0-201-48567-7. OCLC 41017370 (<https://search.worldcat.org/oclc/41017370>).
12. Smart, John Ferguson (2008). *Java Power Tools* (<https://books.google.com/books?id=kE0UDQAAQBAJ&q=visual+sourcesafe+atomic+commit&pg=PA301>). O'Reilly Media, Inc. p. 301. ISBN 978-1-4919-5454-6. Retrieved 26 July 2018.
13. Martins, Luana; Pontillo, Valeria; Costa, Heitor; Ferrucci, Filomena; Palomba, Fabio; Machado, Ivan (2024). "Test code refactoring unveiled: where and how does it affect test code quality and effectiveness?". *Empirical Software Engineering*. **30** (1): 27. doi:10.1007/s10664-024-10577-y (<https://doi.org/10.1007/2Fs10664-024-10577-y>).
14. (these are only about OOP however).Refactoring techniques in Fowler's refactoring Website (<http://refactoring.com/catalog/index.html>)
15. Ferrante, Jeanne; Ottenstein, Karl J.; Warren, Joe D. (July 1987). "The program dependence graph and its use in optimization" (<https://doi.org/10.1145/2F24039.24041>). *ACM Transactions on Programming Languages and Systems*. **9** (3). ACM: 319–349. doi:10.1145/24039.24041 (<https://doi.org/10.1145/2F24039.24041>). S2CID 505075 (<https://api.semanticscholar.org/CorpusID:505075>).
16. Donglin, Linag; Harrold, M. J. (November 2008). "Slicing objects using system dependence graphs". *Proceedings, International Conference on Software Maintenance (Cat. No. 98CB36272)*. IEEE. pp. 319–349. doi:10.1109/ICSM.1998.738527 (<https://doi.org/10.1109/2FICSM.1998.738527>). ISBN 978-0-8186-8779-2. S2CID 18160599 (<https://api.semanticscholar.org/CorpusID:18160599>).
17. "Replace type-checking code with State/Strategy" (<http://refactoring.com/catalog/replaceTypeCodeWithStateStrategy.html>).
18. "Replace conditional with polymorphism" (<http://refactoring.com/catalog/replaceConditionalWithPolymorphism.html>).
19. Bruntink, M.; van Deursen, A.; Tourwe, T.; van Engelen, R. (2004). "An evaluation of clone detection techniques for crosscutting concerns". *Proceedings, 20th IEEE International Conference on Software Maintenance*. IEEE. doi:10.1109/ICSM.2004.1357804 (<https://doi.org/10.1109/2FICSM.2004.1357804>).
20. Hardware description languages#HDL and programming languages
21. Kaiping Zeng, Sorin A. Huss, "Architecture refinements by code refactoring of behavioral VHDL-AMS models". ISCAS 2006
22. M. Keating, "Complexity, Abstraction, and the Challenges of Designing Complex Systems", in DAC'08 tutorial [1] Archived (<https://web.archive.org/web/20160328163412/https://dac.com/events/eventdetails.aspx?id=77-130>) 2016-03-28 at the [Wayback Machine](#)"Bridging a Verification Gap: C++ to RTL for Practical Design"

23. M. Keating, P. Bricaud: *Reuse Methodology Manual for System-on-a-Chip Designs*, Kluwer Academic Publishers, 1999.
24. Opdyke, William F.; Johnson, Ralph E. (September 1990). "Refactoring: An Aid in Designing Application Frameworks and Evolving Object-Oriented Systems". *Proceedings of the Symposium on Object Oriented Programming Emphasizing Practical Applications (SOOPPA)*. ACM.
25. Griswold, William G (July 1991). *Program Restructuring as an Aid to Software Maintenance* (<https://cseweb.ucsd.edu/~wgg/Abstracts/gristhesis.pdf>) (PDF) (Ph.D. thesis). University of Washington. Retrieved 2011-12-24.
26. Opdyke, William F (June 1992). *Refactoring Object-Oriented Frameworks* (<https://dl.acm.org/doi/book/10.5555/169783>) (compressed Postscript) (Ph.D. thesis). University of Illinois at Urbana-Champaign. Retrieved 2008-02-12.
27. "Martin Fowler, "MF Bliki: EtymologyOfRefactoring" " (<http://martinfowler.com/bliki/EtymologyOfRefactoring.html>).
28. Brodie, Leo (2004). *Thinking Forth* (<http://thinking-forth.sourceforge.net>). Fig Leaf Press, Forth Interest. pp. 171–196. ISBN 0-9764587-0-5. Archived (<https://web.archive.org/web/20051216163615/http://thinking-forth.sourceforge.net/>) from the original on 16 December 2005. Retrieved 3 May 2020.
29. Sokolov, Andriy. "What is code refactoring?" (<https://ducode.io/blog/what-is-code-refactoring/>).
30. "What's new in Xcode 9" (https://developer.apple.com/library/content/documentation/DeveloperTools/Conceptual/WhatsNewXcode/xcode_9/xcode_9.html).
31. "Overview | Qt Creator Documentation" (<https://doc.qt.io/qtcreator/creator-overview.html>).

Further reading

- Wake, William C. (2003). *Refactoring Workbook*. Addison-Wesley. ISBN 978-0-321-10929-3.
- Mens, T.; Tourwe, T. (February 2004). "A survey of software refactoring" (<https://zenodo.org/record/848931>). *IEEE Transactions on Software Engineering*. **30** (2): 126–139. Bibcode:2004ITSEn..30..126M (<https://ui.adsabs.harvard.edu/abs/2004ITSEn..30..126M>). doi:10.1109/tse.2004.1265817 (<https://doi.org/10.1109%2Ftse.2004.1265817>). ISSN 0098-5589 (<https://search.worldcat.org/issn/0098-5589>). S2CID 206778272 (<https://api.semanticscholar.org/CorpusID:206778272>).
- Feathers, Michael C (2004). *Working Effectively with Legacy Code*. Prentice Hall. ISBN 978-0-13-117705-5.
- Kerievsky, Joshua (2004). *Refactoring To Patterns*. Addison-Wesley. ISBN 978-0-321-21335-8.
- Arsenovski, Danijel (2008). *Professional Refactoring in Visual Basic*. Wrox. ISBN 978-0-470-17979-6.
- Arsenovski, Danijel (2009). *Professional Refactoring in C# and ASP.NET*. Wrox. ISBN 978-0-470-43452-9.
- Ritchie, Peter (2010). *Refactoring with Visual Studio 2010*. Packt. ISBN 978-1-84968-010-3.

External links

- What Is Refactoring? (<http://c2.com/cgi/wiki?WhatIsRefactoring>) (c2.com article)
- Martin Fowler's homepage about refactoring (<http://refactoring.com/>)

